UFR de mathématique et d'informatique
Université de Strasbourg

BDR THERMEA GROUP

Automatic Model Validation using Pipelines

Ecker Nick

30th January 2024

Supervisors: Christophe Prud'homme

Didier Lemetayer

Contents

1 Introduction

In the realm of thermal systems, heat pumps stand out as a cornerstone of modern heating and cooling solutions. They are pivotal in transitioning towards energy-efficient and environmentally friendly technologies. The efficacy of these systems hinges on the precision and reliability of their underlying models. This is where model validation becomes vital.

Model validation in heat pumps is a rigorous process of ensuring that the computational models accurately represent the real-world behavior of these systems. It involves a meticulous comparison of the model's output with empirical data. The essence of this process lies in its ability to guarantee that the models predict the performance, efficiency, and environmental impact of heat pumps under various conditions with high fidelity.

In an era where sustainability and energy efficiency are paramount, the role of heat pumps is increasingly significant. They are not only a solution for reducing carbon footprints in residential and commercial spaces but also a key element in the larger tapestry of renewable energy systems. Thus, ensuring the accuracy and reliability of heat pump models through validation is not just a technical necessity; it's a step towards a sustainable future.

As we embark on this project with BDR Thermea, our focus will be on developing and refining methodologies for automatic model validation. This will not only streamline the process but also enhance the efficiency and accuracy of model validation in heat pumps. By doing so, we aim to contribute significantly to the development of more efficient, reliable, and sustainable heating and cooling systems.

2 Presentation of BDR Thermea

2.1 Company Overview

BDR Thermea is a renowned leader in the heating and cooling industry, known for its commitment to innovation and sustainability. With a rich history that spans over decades, BDR Thermea has established itself as a key player in providing advanced heating solutions globally. The company's vision is centered around delivering energy-efficient and environmentally friendly heating technologies, reflecting its dedication to tackling modern energy challenges.

2.2 Missions of BDR Thermea

At the heart of BDR Thermea's operations are its core missions: to lead in the transition towards a carbon-neutral future, to innovate in the field of thermal comfort, and to deliver exceptional value to its customers. These missions drive the company's efforts in developing cutting-edge technologies that not only meet current heating and cooling needs but also anticipate future demands in a world increasingly aware of environmental impacts.

2.3 CAE and Modeling Department

The Computer-Aided Engineering (CAE) and Modeling Department at BDR Thermea plays a crucial role in the company's innovation pipeline. This department is instrumental in developing sophisticated models and simulations that are vital for the design and improvement of heating systems.

The department's mission is to drive excellence in engineering through advanced modeling and simulation techniques. Its activities include developing and validating thermal models, conducting performance analyses, and optimizing design processes for new and existing products. The team's expertise in CAE and modeling is a cornerstone in BDR Thermea's commitment to delivering state-of-the-art heating and cooling solutions.

3 Overview of Heat Pumps

3.1 Introduction to Heat Pumps

Heat pumps are energy-efficient devices used for heating and cooling buildings. They operate by transferring heat from a cooler space to a warmer space, using the principles of thermodynamics. This process is reversible, allowing heat pumps to provide heating in winter and cooling in summer. They are considered environmentally friendly because they use less electrical energy compared to traditional heating and cooling systems.

3.2 Components of Heat Pumps

1. **Compressor:** The compressor is the heart of the heat pump system. It circulates the refrigerant through the system and increases the pressure and temperature of the refrigerant gas.

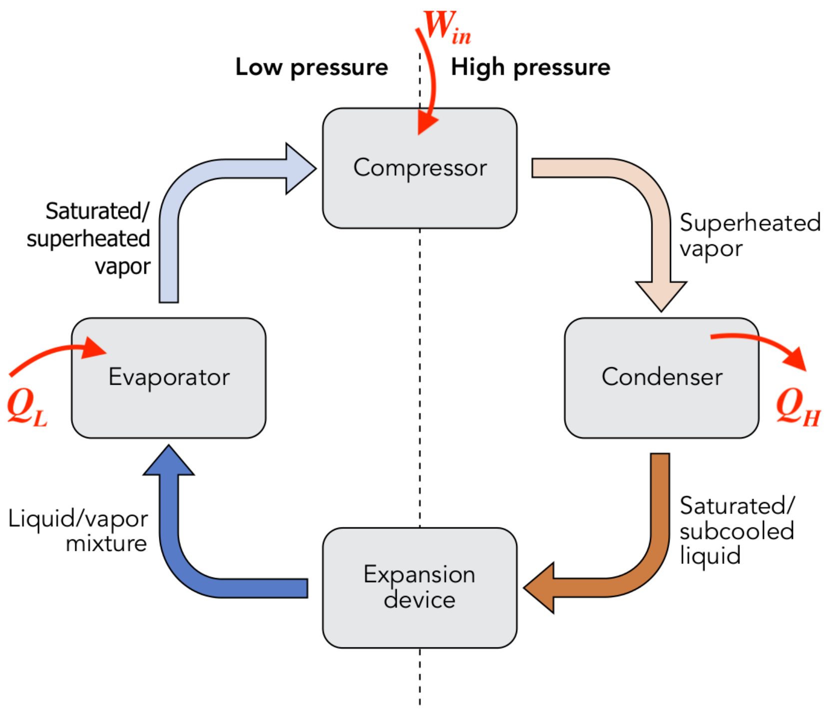
2. **Condenser:** In the condenser, the hot refrigerant gas releases its heat to the surroundings, condensing into a liquid. This heat is then used to warm the building or water in heating applications.

3. **Expansion Valve:** The expansion valve reduces the pressure of the refrigerant liquid, cooling it down rapidly. This is a critical component for regulating the flow of refrigerant and maintaining the efficiency of the system.

4. **Evaporator:** In the evaporator, the cold refrigerant absorbs heat from the external environment (air, ground, or water source). It then evaporates, turning into a low-pressure gas, and is sent back to the compressor to restart the cycle.

3.3 Working Principle

In heating mode, the heat pump extracts heat from external sources (like air, water, or ground) and transfers it indoors. In cooling mode, it reverses the process, removing heat from the interior and releasing it outside. This versatility makes heat pumps an increasingly popular choice for climate control in various settings.



4 Project Overview/Goal of the Project

4.1 Main Objectives

The project's main objectives are designed to streamline the model validation process by leveraging automation and integration techniques. These objectives include:

1. **Development of an Automated Pipeline:**
   - The core task is to create a pipeline that can automatically invoke a software application on an agent pool. This software will execute models that are stored in a Git repository. The automation of this process is crucial for improving efficiency and reducing the potential for human error.
2. **Extension of Pipeline Capabilities:**
   - Beyond basic model execution, the pipeline will be extended to perform more complex tasks. An example of this is the automatic generation of Functional Mock-up Units (FMUs), which are integral to the model val-

idation process. This extension will demonstrate the pipeline's flexibility and capability to handle various aspects of model validation.

3. **Automated Model Validation Methodology:**

   - A key deliverable is to propose and implement a method to automate the model validation process using pipelines. This involves setting up a system that can automatically compare the model outputs with empirical data, ensuring that the model accurately represents real-world scenarios.

4. **Deployment to Other Components:**

   - Finally, the developed methodology and pipeline will be adapted and deployed for other components within the heat pump system. This scalability is essential to ensure that the entire heat pump model, not just individual parts, is validated efficiently and accurately.

4.2 Expected Outcomes

By achieving these objectives, the project is expected to deliver:

- **Enhanced Efficiency:** Automated pipelines significantly reduce the time and effort required for model validation.
- **Increased Accuracy:** Automation minimizes human error, leading to more accurate model validation results.
- **Scalability:** The ability to deploy the method to other components demonstrates the scalability of the solution, making it adaptable to various model types and sizes.

5 Roadmap

1. **Understanding Heat Pumps:**

   - Prior to commencing the technical work, a crucial step in our project plan is to undertake a comprehensive formation on the principles and functioning of heat pumps. This step is deemed essential for gaining an in-depth understanding of the systems that we will be validating.

2. **Analysis of Current Repositories:**
   - The next step involves a thorough analysis of existing repositories. This includes understanding how heat pumps are currently modeled, how these models are stored, and the processes for calling these models. This stage is vital for identifying areas of improvement and integration for the automated pipeline.

3. **Creating the Initial Pipeline:**
   - The primary task is to create a pipeline capable of calling software on an agent pool to execute models located in a Git repository. This step marks the beginning of the automation process in model validation.

4. **Pipeline Extension:**
   - Following the initial setup, the pipeline will be extended to include more complex functionalities, such as automatic FMU generation. This extension is aimed at demonstrating the pipeline's adaptability and comprehensive coverage of the validation process.

5. **Automating Model Validation:**
   - The project will explore various methodologies to automate the model validation process using pipelines. This involves creating a system that can automatically compare the model outputs with empirical data for accurate validation.

6. **Deployment to Other Models:**
   - Finally, the validated method will be deployed across other models, ensuring that the automated validation process is not limited to a single model but is scalable and adaptable to various models within the heat pump system.

The final aim is to establish an automated, accurate, and scalable model validation methodology for heat pumps.

6 Technical Components Presentation

In this section, we will explore the key technical components utilized in the project. These components form the backbone of our automatic model validation framework and are essential in ensuring the efficacy and accuracy of our methodologies.

6.1 Dymola

Dymola (Dynamic Modeling Laboratory) is a comprehensive tool for modeling and simulating complex integrated systems.

Dymola is used for developing detailed physical models using Modelica, a non-proprietary, object-oriented language designed for component-oriented modeling.

It allows for the creation of dynamic simulations of heat pump systems, enabling us to test various operational scenarios and compare the simulated performance with real-world data.

6.2 Thermal Systems

The Thermal Systems library is a crucial component of our project, serving as an integral part of the modeling and simulation environment within Dymola. This library is designed to provide a comprehensive set of tools and models for simulating thermal systems, particularly heat pumps.

The Thermal Systems library offers a wide range of tools and components that are specifically tailored for modeling various aspects of heat pump systems. This includes models for compressors, condensers, expansion valves and evaporators.

Utilizing the library, we develop detailed models of heat pump systems. These models are then used to simulate real-world scenarios, enabling us to analyze the performance of heat pumps under various conditions.

Models developed using the Thermal Systems library serve as benchmarks for validation. We compare these models' outputs with empirical data to ensure that they accurately represent the behavior of actual heat pump systems.

6.3 FMU/FMI

Functional Mock-up Interface (FMI) and Functional Mock-up Unit (FMU) are critical standards in the project:

FMI is an open standard for the exchange and co-simulation of dynamic models, while an FMU is a specific implementation of this standard.

They are used for integrating models developed in Dymola with other simulation tools and environments. This integration is essential for a comprehensive approach to model validation.

The use of FMI/FMU facilitates the consistent and accurate exchange of simulation data, ensuring that our models remain compatible and interoperable with different systems and tools, which is vital for a collaborative project like ours.

6.4 YAML Pipelines

YAML (YAML Ain't Markup Language) pipelines are an integral part of the automation process:

These pipelines are used to define the sequences of steps that the model validation process must follow, from data ingestion to simulation and analysis.

In our project, YAML pipelines are configured to automate various stages of model validation, ensuring consistency and efficiency.

6.5 Azure DevOps

Azure DevOps represents a suite of development tools provided by Microsoft, designed to support a comprehensive range of software development needs from planning and project management to code development and deployment. One of the critical features of Azure DevOps is its Pipelines, which offer a service for automating builds, tests, and deployments. Pipelines enable continuous integration and continuous delivery (CI/CD) practices, allowing teams to frequently update applications in a reliable manner.

In the context of our project with BDR Thermea, we aim to leverage Azure DevOps Pipelines to automate the process of model validation. This involves configuring pipelines to automatically execute Dymola commands for converting '.mo' files into '.fmu' format and validating these models against empirical data.

7 Achieved Goals

7.1 Formation on Heat Pumps

The first and pivotal task of the project was a thorough familiarization with the principles and functioning of heat pumps, achieved through a dedicated formation. This foundational step was instrumental in equipping us with an in-depth understanding of the systems we are set to model and validate. Completing this initial phase also gave me an understanding how we can calculate the energy produced by a heat pump. This step has been essential in ensuring that our technical work is informed and enhanced by a comprehensive knowledge of the core subject matter, to ensure a solid groundwork for the next stages of our project.

7.2 Analyzing the current repositories

The next step of the project was to thouroughly analyze the repositories of BDRThermea which contained the Modelica files for their different models, which are also used to perform their simulations.

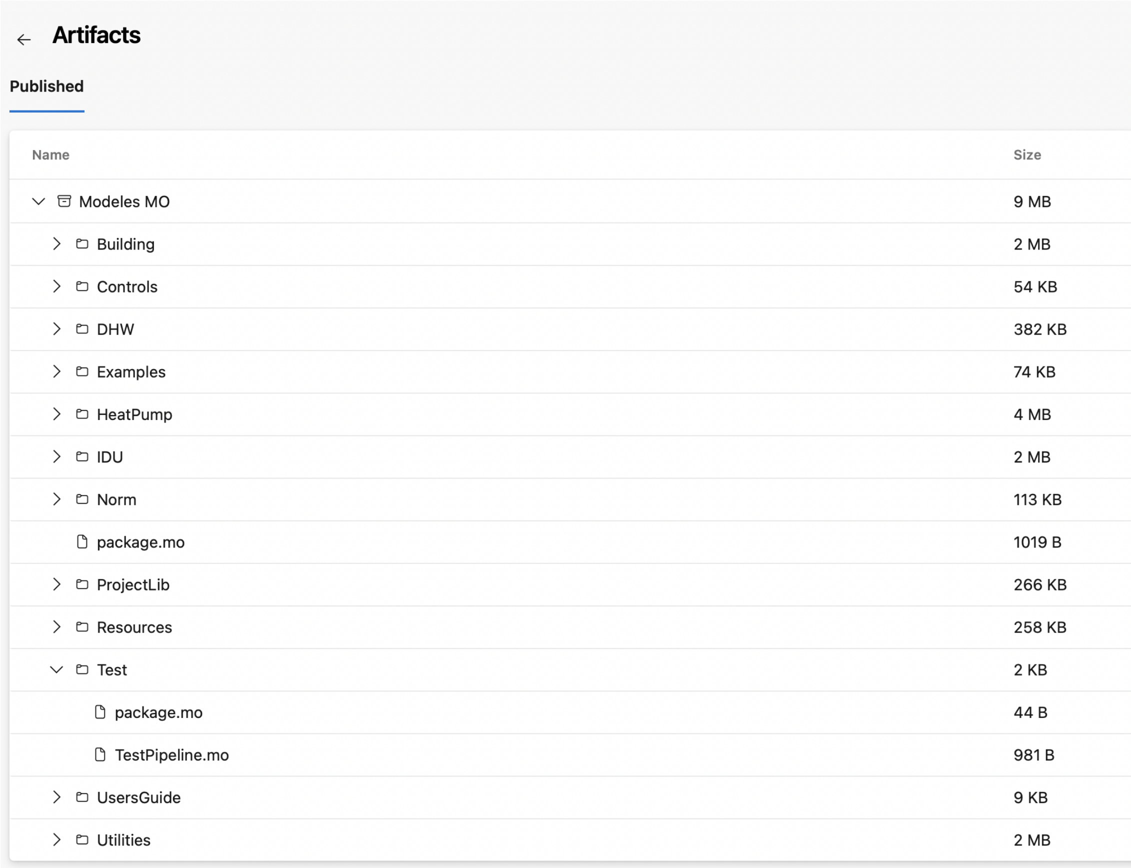
To ensure this task, I successfully utilized Azure DevOps pipelines, executed through .yml files, to perform a series of tasks essential for analyzing our repositories. Each task, completed with specific commands, contributed significantly to our project's progress.

- Listing .mo Files in a Specific Directory **Command Used:** `dir /s /b *.mo` This command was employed to identify and list all files with the .mo (Modelica) extension in the directory `HPSiMoLib`. It allowed for a comprehensive overview of the available Modelica models.

  Here is the pipelines with a basic architecture allowing to identify all .mo files and gather them in an output:

```yaml
# Pipeline pour lister les fichiers .mo
trigger:
- master
# Le pipeline se déclenche dès qu'il y a un changement sur la branche master

pool:
  vmImage: windows-latest
  name: BDUHPSimulation

steps:
# Cette étape liste tous les fichiers .mo dans le répertoire HPSiMoLib
- script: dir /s /b *.mo
  workingDirectory: HPSiMoLib
  displayName: 'Lister les fichiers .mo'

# Copie des fichiers .mo dans un répertoire Output sur l'agent
- task: CopyFiles@2
  inputs:
    SourceFolder: 'HPSiMoLib'
    Contents: '**/*.mo'
    TargetFolder: 'Output'
  displayName: 'Copier les fichiers .mo dans Output'

# Publication du répertoire Output comme artefact
- task: PublishBuildArtifacts@1
  inputs:
    PathtoPublish: 'Output'
    ArtifactName: 'Modeles MO'
    publishLocation: 'Container'
```

  Here is the according output, keeping the architecture of the repositories:



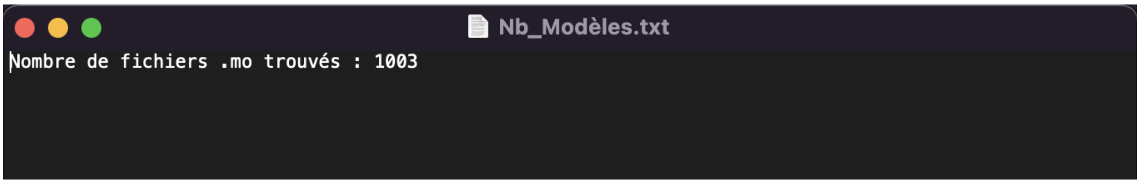
- Counting the Number of Specific .mo Files

  Command Used: PowerShell script with `Get-ChildItem` and `Count`
  The pipeline used a PowerShell script to count .mo files. This operation was used to determine the number of .mo files present in the organisation and can be combined to search for files with a specific name.

  Here are the core lines of the .yml file allowing us to perform this operation:

```
# Compte le nombre de fichiers .mo et affiche le résultat en utilisant PowerShell
- powershell: |
    Write-Host "Comptage des fichiers .mo dans HPSiMoLib..."
    $count = (Get-ChildItem -Recurse -Filter *.mo).Count
    Write-Host "Nombre de fichiers .mo trouvés : $count"
    $output = "Nombre de fichiers .mo trouvés : $count"
    $output | Out-File -FilePath Nb_Modèles.txt -Encoding UTF8
  workingDirectory: HPSiMoLib
  displayName: 'Compter et enregistrer le nombre de fichiers .mo'

# Copie des fichiers .mo dans un répertoire Output sur l'agent
- task: CopyFiles@2
  inputs:
    SourceFolder: 'HPSiMoLib'
    Contents: |
      **/*.mo
      Nb_Modèles.txt
    TargetFolder: 'Output'
  displayName: 'Copier les fichiers .mo et Nb_Modèles.txt dans Output'
```

When counting the number of .mo files in the whole group of repositories and saving the number in a text file, this is our result:


Nb_Modèles.txt

Nombre de fichiers .mo trouvés : 1003

- Finding Files Containing a Specific String

  Command Used: PowerShell script with `Get-ChildItem` and `Select-String`
  This part of the pipeline identified the files containing a specific name. Using this, we can very precisely extract a model when having it's name.

- Searching for a Specific Directory

  Command Used: `dir /s /b /ad "validated model"`
  Using this command in a script performs the search for a directory named "validated model". This task can be used to verify the existence and location of key directories related to model validation processes.

  Here the core lines in the .yml file:

```
steps:
# Cette étape recherche le répertoire "validated model" dans le répertoire HPSiMoLib et ses sous-répertoires
- script: |
    dir /s /b /ad "validated model"
  workingDirectory: HPSiMoLib
  displayName: 'Rechercher le répertoire "validated model"'
```

8 Failed Task: Executing Dymola Commands in Pipeline

To integrate Dymola as a variable in a pipeline, we used the following lines:

```
variables:
  Dymola: '"C:\Program Files\Dymola 2021x\bin64\Dymola.exe"'
```

During the course of the project, we encountered a significant hurdle with the use of Dymola in our Azure DevOps pipeline. This unexpected issue, centered around the execution of Dymola commands, effectively impeded our ability to progress with key tasks and objectives.

This error occured as soon as we wanted to use the Dymola variable for a concrete task, like for the following command that should convert a .mo file into a .fmu file:

```
steps:
# Running a Dymola script to export .mo model to .fmu
- script: |
    $(Dymola) -s "translateModelFMU('TestPipeline_FMU', false, 'TestFMU', '2', 'csSolver', true, 0)"
```

8.1 Intended Tasks with Dymola Commands

The pipeline was designed to automate several critical tasks using Dymola, with each task having significant implications for our project's progress:

1. **Exporting Modelica Models to FMU:** The primary task was to use Dymola's 'translateModelFMU' command to convert '.mo' files into '.fmu' format. This conversion was vital for making the models usable in diverse simulation environments and was intended to:
   - Specify the model to be converted (e.g., 'TestPipeline_FMU').
   - Set parameters such as FMU version, solver type, and state handling.
   - Generate an '.fmu' file conforming to the FMI standard.
2. **Automation of Model Conversion Process:** Automating the process of converting Modelica models to FMU format was expected to streamline collabo-

rative projects, enable model sharing, and facilitate cross-platform simulations, thus enhancing interoperability and the utility of the developed models.

8.2 Challenges with Dymola Variable in Pipeline

However, upon integrating Dymola as a variable in the .yml file and attempting to execute these commands, the pipeline faced a critical issue:

Stagnation of Pipeline: Every operation that utilized the Dymola variable resulted in the pipeline becoming stuck. The specific challenges included:

- The pipeline failed to complete the current task or advance to the next one whenever a Dymola command was executed.
- Potential causes identified included Dymola awaiting the verification of a valid licence.
- Efforts to run Dymola in batch mode and implement error handling did not resolve the issue.

Impact: This stagnation posed a significant obstacle in our project, hindering the automation of all the crucial tasks that needed Dymola.

Potential Licensing Issue:
Colleagues at BDR Thermea have hypothesized that the root of the problem with Dymola's functionality in the pipeline might be related to licensing. It is believed that Dymola is awaiting validation of a proper license before it can execute the commands within the pipeline environment. This theory, however, has not yet been empirically tested and remains speculative. As such, there is a need for comprehensive analysis and testing to confirm whether the licensing issue is indeed the cause of the pipeline's stagnation.

Verifying Licensing as a Potential Solution:
To ascertain whether licensing is indeed the cause of the issue with Dymola in our pipeline, a proposed solution is to integrate commands within the pipeline that specifically retrieve and verify a license from BDR Thermea's license server. This step would be executed prior to utilizing any Dymola commands. By incorporating

such a mechanism, we could ensure that Dymola has the necessary authorization for operation in an automated environment. This approach would not only serve as a test to verify the licensing hypothesis but also potentially rectify the problem if a valid license is indeed required.

9 Post-Resolution steps of Dymola Command Issue

The unexpected challenges encountered with Dymola in our pipeline have led to a temporary halt in the project's advancement. However, once this issue is resolved, we have a clear plan for the next steps, crucial for the successful validation of our models and the overall progress of the project.

9.1 Model Conversion Using Dymola

The immediate next step, once the pipeline is fully operational, involves using the `translateModelFMU` command in Dymola. This command will allow us to convert a chosen '.mo' file into an '.fmu' file. The '.fmu' file format is essential for its compatibility with a wide range of simulation tools, thus facilitating the validation process.

9.2 Model Validation with Empirical Data

For the subsequent step of model validation:

1. **Gathering Empirical Data:** We plan to acquire empirical data from a heat pump, which BDR Thermea can provide from their operational setups. This real-world data is critical for benchmarking our model's performance.
2. **Simulating and Comparing:** The next phase involves performing simulations on the '.fmu' file, using conditions and parameters that match those under which the empirical data was gathered. By aligning the simulation conditions with the empirical data, we aim to achieve a direct comparison between the

two.

3. **Error Calculation:** We will employ a tailored method to calculate the error margin between the simulation results and the empirical data. This step is vital for quantitatively assessing the accuracy of our model. It is important to note that the precise methodology for quantifying the error will be determined only after we are able to obtain and analyze the results from the simulations. This ensures that our approach to error quantification is most suitable and effective based on the actual simulation outcomes.

9.3 Extending Validation to Other Models

Once we have successfully validated a model (e.g., for a specific heat pump), our goal is to apply the validation method to other models within the heat pump system. This approach ensures that our validation method is robust and applicable across different models, thereby guaranteeing the reliability and consistency of our validation process.

10 Possible Advancements After Finishing the Project

Upon successful completion of our project and the validation of our models, there are several potential directions for further advancement. These directions leverage the validated models for a variety of applications, enhancing their utility and impact:

- **Parameter Optimization:** Optimize the parameters of the models to enhance performance and efficiency. This involves using algorithms to fine-tune model parameters for optimal operation under various conditions.
- **Scalability Testing:** Test the models on larger scales or in different types of heat pump systems. This would assess the robustness and adaptability of the models in varied operational contexts.
- **Uncertainty Analysis:** Conduct an in-depth uncertainty analysis of the model parameters. This analysis would quantify the confidence in the model

predictions and identify which parameters significantly impact the model's output.

- **Predictive Maintenance Modeling:** Develop methods for predictive maintenance to anticipate potential failures or maintenance needs. This application could significantly improve operational reliability and reduce downtime.

These potential advancements represent an opportunity to extend the utility and application of our models well beyond the initial scope of the project, offering new avenues for exploration and development in the field of heat pump systems.

11 Conclusion

To conclude this report, it's important to reflect on the journey of our project. Our work began with the goal of automating the validation of models for heat pumps, a critical component in enhancing the efficiency and sustainability of thermal systems. The project aimed to streamline the model validation process through the development of an automated pipeline, leveraging the capabilities of Azure DevOps and the dynamic modeling power of Dymola.

Throughout the project, we achieved several significant milestones. We familiarized ourselves thoroughly with the principles and functioning of heat pumps, which laid a solid foundation for our technical work. We successfully utilized Azure DevOps pipelines to analyze existing repositories, identifying and listing Modelica files, counting specific '.mo' files, finding files containing certain strings, and locating key directories. These steps were pivotal in setting the stage for the subsequent phases of the project.

However, despite the progress and achievements, we encountered a major problem that halted further advancement. The integration of Dymola as a variable in our pipeline, essential for automating the conversion of '.mo' files to '.fmu' format, faced unexpected challenges. The pipeline experienced stagnation, failing to complete tasks or advance whenever a Dymola command was executed.

The unresolved issue with Dymola has, unfortunately, prevented me from completing the project as initially envisioned. This setback underscores the complexities and unpredictability inherent in integrating advanced modeling tools into automated pipelines.

In conclusion, the project's objectives were not fully met, primarily due to the unforeseen challenges encountered with Dymola and the delayed access to the Azure DevOps repositories. The latter significantly restricted my ability to make early progress in the project. Despite these setbacks, the foundational work completed has set the stage for the following steps. With additional time and a resolution to the Dymola issue, there is confidence that the project can be successfully fulfilled.

References

[1] BDR Thermea Group. (2023). *About Us*. Available at: `https://www.bdrthermeagroup.com/en/about-us/`

[2] Ben-Kiki, O., Evans, C., & döt Net, I. (2009). *YAML Ain't Markup Language (YAML™) Version 1.2*. Available at: `https://yaml.org/spec/1.2/spec.html`

[3] Kumar, A., & Kolar, P. (2016). *Automation in model validation of multi-physical systems using Modelica*. In 2016 IEEE International Systems Engineering Symposium (ISSE). Available at: `https://ieeexplore.ieee.org/abstract/document/7745467`

[4] Functional Mock-up Interface (FMI). *FMI Standard*. Available at: `https://fmi-standard.org`

[5] Wikipedia contributors. *Heat Pump*. Available at: `https://en.wikipedia.org/wiki/Heat_pump`

[6] Claytex. *Thermal Systems Library for Dymola*. Available at: `https://www.claytex.com/products/dymola/model-libraries/thermal-systems-library/`

[7] BCcampus. *6.2 Refrigerator and Heat Pump*. Available at: `https://pressbooks.bccampus.ca/thermo1/chapter/6-2-refrigerator-and-heat-pump/`

[8] OpenModelica *OpenModelica Documentation*. Available at: `https://openmodelica.org/doc/OpenModelicaUsersGuide/v1.11.0/`

[9] Dassault Systems *Dymola Developers*. Available at: `https://www.3ds.com/fr/produits-et-services/catia/produits/dymola/`